

# HTML SVG

A Practical, From-Zero Learning Guide

## Part 1 — Foundations, Coordinate Systems & Basic Shapes

This series is designed to take you from writing your first SVG by hand to building icons, illustrations, charts, animations and interactive graphics inside HTML. Examples are intentionally small so you can type and modify them yourself.

**How to use this PDF:** Read a concept → type the example yourself → change 2–3 values → observe what happens → complete the exercises.

# 1. What is SVG?

**SVG** means **Scalable Vector Graphics**. It is an XML-based markup language for describing two-dimensional graphics. In HTML, SVG can be written directly inside the page.

Unlike a raster image such as JPG or PNG, an SVG describes shapes mathematically. That means you can enlarge an SVG without the edges becoming pixelated.

## SVG is especially useful for:

- Logos and brand marks
- Icons and UI illustrations
- Diagrams and technical drawings
- Charts and data visualization
- Maps
- Animated graphics
- Interactive graphics controlled with JavaScript

# 2. Your First SVG

The basic container is the **<svg>** element.

```
<!DOCTYPE html>
<html>
<body>

<svg width="300" height="200">
  <circle cx="100" cy="100" r="50" />
</svg>

</body>
</html>
```

Here, **width** and **height** control the displayed SVG area. **circle** creates a circle. The attributes **cx**, **cy**, and **r** control its center and radius.

# 3. SVG Coordinates

SVG normally starts its coordinate system at the **top-left corner**. The x-axis increases to the right, and the y-axis increases downward.

```
(0,0) -----> x
|
|
|
v
y
```

So a point at **x=100, y=50** is 100 units from the left and 50 units from the top.

# 4. The viewBox — One of the Most Important Concepts

The **viewBox** defines the internal coordinate system of an SVG. It is written as four numbers: **min-x min-y width height**.

```
<svg width="400" height="300"
  viewBox="0 0 400 300">
```

```

    <circle cx="200" cy="150" r="80"
            fill="tomato" />
</svg>

```

The displayed size can be different from the viewBox. This is what makes SVGs so powerful for responsive graphics.

```

<svg width="100%" viewBox="0 0 400 300">
  ...
</svg>

```

A useful mental model: **width/height** = **display size**, while **viewBox** = **drawing coordinate system**.

## 5. Styling SVG

SVG shapes can be styled with presentation attributes, CSS, or both. The most common properties are **fill**, **stroke**, and **stroke-width**.

```

<svg width="250" height="150" viewBox="0 0 250 150">
  <rect x="30" y="30" width="190" height="90"
        fill="skyblue"
        stroke="black"
        stroke-width="4" />
</svg>

```

You can also use CSS:

```

<style>
  .box {
    fill: #60a5fa;
    stroke: #111827;
    stroke-width: 4;
  }
</style>

<svg viewBox="0 0 250 150">
  <rect class="box" x="30" y="30"
        width="190" height="90" />
</svg>

```

## 6. Basic Shape: Rectangle

```

<rect
  x="20"
  y="30"
  width="120"
  height="70"
  rx="10"
  ry="10"
/>

```

**x** and **y** set the top-left position. **width** and **height** set its size. **rx** and **ry** round the corners.

## 7. Basic Shape: Circle

```

<circle
  cx="120"
  cy="80"
  r="50"
  fill="orange"
/>

```

**cx** and **cy** define the center. **r** defines the radius.

## 8. Basic Shape: Ellipse

```
<ellipse
  cx="120"
  cy="80"
  rx="80"
  ry="45"
  fill="purple"
/>
```

**rx** is the horizontal radius and **ry** is the vertical radius.

## 9. Basic Shape: Line

```
<line
  x1="20"
  y1="30"
  x2="220"
  y2="120"
  stroke="black"
  stroke-width="5"
/>
```

A line uses a start point (**x1,y1**) and an end point (**x2,y2**).

## 10. Basic Shape: Polyline

```
<polyline
  points="20,120 70,40 120,100 170,30 220,90"
  fill="none"
  stroke="black"
  stroke-width="4"
/>
```

A **polyline** connects multiple points but does not automatically close the shape.

## 11. Basic Shape: Polygon

```
<polygon
  points="120,20 210,80 175,180 65,180 30,80"
  fill="gold"
  stroke="black"
  stroke-width="4"
/>
```

A **polygon** is similar to a polyline, but SVG closes the final segment back to the first point.

## 12. Stroke Properties

Strokes can be customized beyond simply setting a width.

```
<path
  d="M 20 100 L 220 100"
  fill="none"
  stroke="black"
  stroke-width="8"
  stroke-linecap="round"
/>
```

**Important stroke properties include:**

- **stroke-width** — thickness
- **stroke-linecap** — shape of line ends: butt, round, square
- **stroke-linejoin** — shape of corners: miter, round, bevel
- **stroke-dasharray** — creates dashed patterns

- **stroke-dashoffset** — shifts the dash pattern

## 13. Fill vs Stroke

```
<circle
  cx="80" cy="80" r="50"
  fill="lightgreen"
  stroke="darkgreen"
  stroke-width="6"
/>
```

**fill** controls the inside of a closed shape. **stroke** controls its outline. For open shapes such as a line, fill usually has no visible effect.

## 14. opacity and fill-opacity

```
<circle cx="70" cy="80" r="50"
  fill="red" fill-opacity="0.5" />

<circle cx="140" cy="80" r="50"
  fill="blue" opacity="0.5" />
```

**opacity** affects the whole element, including its stroke. **fill-opacity** affects only the fill.

## 15. Your First Mini Project — Traffic Light

```
<svg width="180" height="360"
  viewBox="0 0 180 360">

  <rect x="20" y="10"
    width="140" height="340"
    rx="25"
    fill="#222" />

  <circle cx="90" cy="80"
    r="40"
    fill="red" />

  <circle cx="90" cy="180"
    r="40"
    fill="gold" />

  <circle cx="90" cy="280"
    r="40"
    fill="limegreen" />

</svg>
```

Try changing the circle radius, box corner radius, colors, and spacing. Then add a fourth circle and redesign the traffic light yourself.

## 16. Practice Exercises

- Draw a blue rectangle with rounded corners.
- Draw a circle with only a thick outline and no fill.
- Create a five-sided polygon.
- Create a zig-zag line using polyline.
- Make a dashed horizontal line.
- Create a simple house using a rectangle and polygon.

- Create a smiley face using circles and an arc/curve later.
- Make the same SVG responsive using viewBox and width='100%'.

## 17. What Comes Next

Part 2 will move beyond basic shapes into the most important SVG skill: **<path>**. You will learn path commands such as **M, L, H, V, Z, C, S, Q, T, A**, relative commands, curves, and how SVG drawing tools actually represent shapes.

**Suggested learning order:** Foundations → Paths → Groups & transforms → Gradients & patterns → Text → Masks & clip paths → Filters → Symbols/use → ViewBox & responsive SVG → Animation → JavaScript interaction → SVG projects.

## Quick Reference — Part 1

| Element      | Purpose                       |
|--------------|-------------------------------|
| <svg>        | SVG drawing container         |
| <rect>       | Rectangle / rounded rectangle |
| <circle>     | Circle                        |
| <ellipse>    | Ellipse                       |
| <line>       | Straight line                 |
| <polyline>   | Connected open points         |
| <polygon>    | Closed polygon                |
| fill         | Interior paint                |
| stroke       | Outline paint                 |
| stroke-width | Outline thickness             |
| viewBox      | Internal coordinate system    |